Fiche de révision officielle & Synthèse d'examen — Initiation au développement (C++)

Thème principal : Conception algorithmique rigoureuse, tripartition, passage par référence, structures répétitives et tableaux statiques

Synthèse exécutive : Ce cours pose les fondements indispensables du développement en C++ selon les exigences académiques de l'IUT de Bayonne et du Pays Basque. Il détaille la tripartition universelle d'Etcheverry (Spécification, Déclarations, Traitement), la distinction fondamentale entre le passage par valeur et par référence (&), l'utilisation des boucles bornées et non-bornées, ainsi que la manipulation sécurisée des tableaux statiques et de leurs algorithmes canoniques (tri par sélection et recherche dichotomique).

La Tripartition Universelle d'Etcheverry & Spécification

Chaque sous-programme en C++ doit être rigoureusement structuré en trois phases distinctes. Cette approche garantit la lisibilité, la maintenabilité et la correction du code. La première phase consiste en une spécification textuelle claire décrivant le but de la fonction, ses données en entrée et ses résultats en sortie. La seconde phase regroupe l'ensemble des déclarations de variables locales, chacune étant commentée quant à son rôle exact. Enfin, la troisième phase contient le traitement algorithmique pur, sans déclaration cachée au milieu des instructions.

- En-tête de spécification formelle (But, Entrées, Sorties)
- Déclarations locales groupées en tête de bloc avec commentaires de rôle
- Traitement algorithmique pur et découplé

Cas concret enseignant : *M. Etcheverry pénalise lourdement toute fonction qui mélange déclarations et calculs ou qui omet la spécification But/Données/Résultats.*

Passage de Paramètres : Valeur vs Référence (&)

En C++, le mode de transmission des arguments à une fonction détermine si la variable originale peut être modifiée. Par défaut, le passage par valeur (ex: int n) duplique la donnée dans la pile d'exécution, protégeant l'appelant. À l'inverse, le passage par référence (ex: int& n) transmet un pointeur implicite (un alias), permettant à la fonction de modifier directement la variable d'origine, ce qui est indispensable pour des fonctions comme le swap ou l'accumulation multiple.

- Passage par valeur : copie locale, pas d'effet de bord
- Passage par référence (&) : alias vers l'adresse mémoire originale
- Optimisation const Type& pour les structures volumineuses

Cas concret enseignant : *Piège d'examen classique : oublier le symbole & dans la fonction swap(int& a, int& b) : les variables échangent leurs valeurs dans la fonction mais restent inchangées dans le main !*

Structures Répétitives & Gestion de Sentinelle

Le choix de la structure répétitive dépend de la nature du problème. La boucle for est dédiée aux parcours bornés (comme le balayage d'un tableau de taille fixe). La boucle while est une boucle à pré-condition idéale pour les recherches ou les arrêts sur sentinelle (par exemple, s'arrêter quand l'utilisateur entre 999). La boucle do-while garantit une exécution minimale et s'impose pour la validation de saisie dans un intervalle fermé.

- Boucle for pour un nombre d'itérations connu à l'avance
- Boucle while pour un arrêt sur événement ou sentinelle
- Boucle do-while pour une saisie contrôlée obligatoire

Cas concret enseignant : *Ne jamais utiliser une boucle infinie avec un break sauvage à l'intérieur : toujours exprimer la condition d'arrêt proprement dans l'en-tête du while.*

Tableaux Statiques & Algorithmes Canoniques

Un tableau statique possède une taille fixe figée à la compilation. Ses éléments sont stockés dans des cases mémoires contiguës accessibles par des indices entiers non-signés allant de 0 à TAILLE - 1. Les algorithmes canoniques associés incluent la recherche séquentielle, le calcul d'agrégats (somme, moyenne), le tri par sélection par recherche de minimums successifs, et la recherche dichotomique, qui requiert obligatoirement un tableau préalablement trié.

- Indices stricts de 0 à taille-1 avec unsigned int
- Tri par sélection (complexité $O(n^2)$)
- Recherche dichotomique (complexité O(log n))

Cas concret enseignant : *Accéder à tab[n] sur un tableau de taille n provoque un comportement indéfini (Segmentation Fault ou corruption silencieuse de la pile mémoire).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas retourner deux valeurs dans une fonction C++ ?
Réponse du professeur : Une fonction en C++ (comme en mathématiques) ne possède qu'un seul point de retour via le mot-clé return. Pour renvoyer plusieurs résultats, il faut soit utiliser des paramètres de sortie passés par référence (&), soit structurer les données dans un enregistrement (struct).

Question étudiant : Quelle est la condition obligatoire pour pouvoir appliquer une recherche dichotomique ?
Réponse du professeur : Le tableau doit impérativement être trié (par ordre croissant ou décroissant). Si le tableau est désordonné, la division par intervalle de la dichotomie mènera à des résultats erronés.

■ Exercice & Démonstration d'Application

Énoncé : Écrire la fonction de calcul de la moyenne des éléments positifs d'un tableau statique de 5 entiers.

Résolution pas à pas :
1. Appliquer la tripartition : rédiger la spécification (But, Données, Résultats). 2. Déclarer les variables locales nécessaires (somme, compteur, indice de boucle). 3. Parcourir le tableau avec une boucle for bornée. 4. Tester si l'élément courant est strictement positif ; si oui, l'ajouter à la somme et incrémenter le compteur. 5. Gérer le cas particulier où aucun élément positif n'est présent pour éviter une division par zéro (retourner -1.0). 6. Effectuer une conversion de type (cast) pour obtenir un résultat flottant précis.

Résultat : double calculerMoyennePositifs(const int tab[], unsigned int nbElts) { int somme = 0; unsigned int compteur = 0; unsigned int i; for (i = 0; i < nbElts; ++i) { if (tab[i] > 0) { somme += tab[i]; compteur++; } } if (compteur == 0) return -1.0; return static_cast(somme) / compteur; }

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre cin >> val et getline(cin, chaine) ?

R : cin >> val s'arrête au premier espace rencontré et ne lit qu'un mot ou une valeur typée, tandis que getline(cin, chaine) lit toute la ligne y compris les espaces jusqu'au saut de ligne.

Q : Quel mot-clé garantit qu'une variable ne sera jamais modifiée après son initialisation ?

R : Le mot-clé const.

Q : Dans quelle phase de la tripartition doit-on déclarer les variables d'une fonction ?

R : Elles doivent toutes être déclarées au tout début du bloc de la fonction, juste après l'en-tête de spécification et avant le traitement algorithmique.